ROS Lab 2: Two-Drone Coordinate Transforms and Cloud Validation

Course Laboratory Report

Abstract— This report implements and tests the MIT VNAV Lab 2 two-drone transformation exercise. The course's ROS 1 Noetic package was built in an Ubuntu 20.04 environment on the provided cloud host, then exercised with turtlesim, static and dynamic tf, RViz, and five visualization markers. Thirty live relative-pose samples satisfied the derived plane and ellipse equations to numerical precision. An equivalent ROS 2 Humble package was also built and tested on the host. The second vehicle follows a parabolic arc in the world x–z plane but an ellipse in the first vehicle's body frame.

I. Scope and Environment

The course slides prescribe Ubuntu 20.04 with ROS Noetic, starting roscore, verifying node communication with turtlesim, and completing the MIT tf and mathematical exercises [1], [2]. The ROS 1 source is based on the MIT-SPARK VNAV2023 tag [3]; the separately supplied cloud implementation is based on the repository's ROS 2 main branch [4]. Their message and build APIs differ, but both encode the same world poses and relative-pose queries. This report uses the IEEE Robotics and Automation Society's ieeeconf template [5] as an ICRA-style course-report layout.

The ROS 1 reproduction commands are in lab2/RUN.md; on Ubuntu 20.04 they require Noetic [6]. The cloud host itself ran Ubuntu 22.04.3 with ROS 2 Humble. To execute the course version, we created an Ubuntu 20.04 Focal root filesystem on that host, installed ROS Noetic and the required build tools there, and built the package with catkin build (also checked with catkin_make). The supplementary native-host path in lab2/ros2_ws/RUN.md uses colcon build and ros2 launch. Both turtlesim controllers publish geometry_msgs/Twist at 20 Hz with linear $x = 1$ and angular $z = 1$. The two-drone scene visualizes idealized poses; it does not simulate flight dynamics or command quadrotor motors.

II. ROS Graph and Implementation

A. Static and Dynamic Graphs

With static:=true, the course's two_drones.launch starts /av1broadcaster and /av2broadcaster, each publishing a fixed transform from world on /tf_static. It also starts /plots_publisher_node, which subscribes to tf and publishes mesh and trail markers on /visuals, and /rviz, which subscribes to both. The ROS 2 cloud launch has the same graph except that its visualizer is /rviz2_node. AV1 starts at $(1, 0, 0)$ and AV2 at $(0, 0, 1)$. Ignoring tf listener helpers, these are the four primary launch nodes. The meshes originate from /visuals, not from the static broadcasters. Figure 1 summarizes the observed frame and marker relationships.

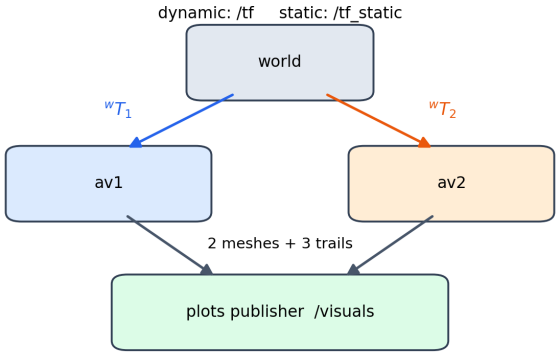


Fig. 1. The world-to-drone tf tree and marker flow. The same edges use /tf_static in the static run and /tf in the dynamic run. The plot node publishes two meshes and three trails on /visuals.

As requested by the 2023 exercise, the ROS 1 static graph can be started manually after roscore, with each command in a separate sourced terminal:

```
rosrun tf2_ros static_transform_publisher \
  1 0 0 0 0 0 1 world av1
rosrun tf2_ros static_transform_publisher \
  0 0 1 0 0 0 1 world av2
rosrun two_drones_pkg plots_publisher_node
rosrun rviz rviz -d $(rospack find \
  two_drones_pkg)/config/default.rviz
```

Without static:=true, the launch files' if branch excludes the static broadcasters and the unless branch starts /frames_publisher_node. That node publishes changing world→av1 and world→av2 poses on /tf. The plot node produces world-frame traces for both vehicles and an AV2-in-AV1 trace. Setting the RViz fixed frame to av1 makes AV1 appear stationary.

B. Pose Publication and Lookup

Let t be seconds since the publisher started. The assignment specifies

$$^w\mathbf{o}_1 = (\cos t, \sin t, 0)^T, \quad ^w\mathbf{o}_2 = (\sin t, 0, \cos 2t)^T. \quad (1)$$

AV1 has zero roll and pitch and yaw t, so its y axis is $(-\sin t, \cos t, 0)^T$, tangent to its circular path. AV2 retains the world orientation. Both implementations stamp TransformStamped messages with the same ROS time, use tf2::Quaternion::setRPY(0,0,t) for AV1, and broadcast both transforms at 50 Hz. Their plot

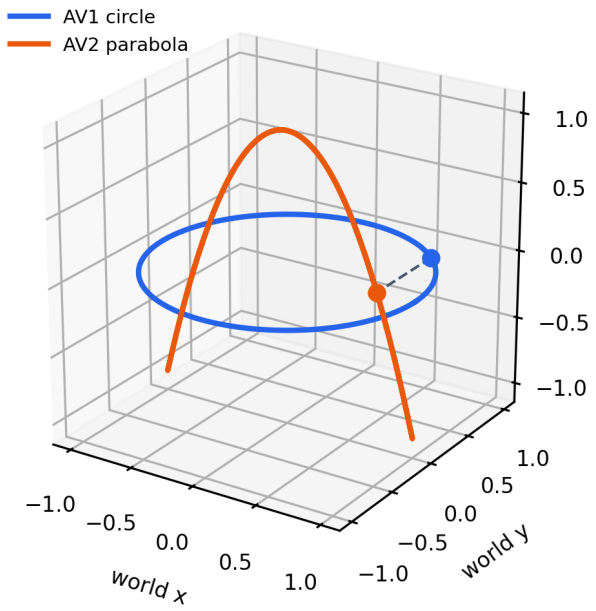


Fig. 2. Analytical three-dimensional world trajectories. Dots mark both poses at $t = \pi/4$; the dashed segment joins them. This is computed from Eq. (1), not an RViz screenshot.

nodes query the latest transform with lookupTransform(ref_frame,dest_frame,...), which returns the destination pose expressed in the reference frame. The ROS 1 listener constructs its tf buffer first and uses a bounded timeout; the ROS 2 listener catches unavailable-frame exceptions during startup.

III. Mathematical Derivations

A. World-Frame Parabola and Relative Pose

For AV2, $x = \sin t$, $y = 0$, and $z = \cos 2t = 1 - 2\sin^2 t = 1 - 2x^2$. Because $x \in [-1, 1]$, the world path is the arc $z = 1-2x^2$ in the x–z plane, traversed repeatedly. Figure 2 shows its spatial relation to AV1's circle.

Write $c = \cos t$ and $s = \sin t$. With ${}^{w}R_1 = R_z(t)$ and ${}^{w}R_2 = I$, the homogeneous transforms and composition are

$$ {}^{w}T_1 = \begin{bmatrix} c & -s & 0 & c \\ s & c & 0 & s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{2} $$

$$ {}^{w}T_2 = \begin{bmatrix} 1 & 0 & 0 & s \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3} $$

Their composition is

$$ {}^{1}T_2 = ({}^{w}T_1)^{-1}{}^{w}T_2 = \begin{bmatrix} c & s & 0 & cs-1 \\ -s & c & 0 & -s^2 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{4} $$

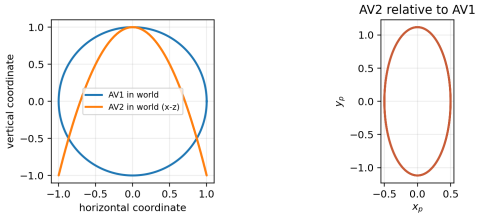


Fig. 3. Analytical world-frame trajectories (left) and AV2 relative to AV1 in the plane coordinates (right). The two curves on the left use different world-coordinate planes and are shown together only for comparison.

Consequently, ${}^{1}\mathbf{o}_2 = (cs - 1, -s^2, \cos 2t)^T$. The plane relation follows from $-s^2 = (\cos 2t - 1)/2$:

$$ \Pi: \quad z_2^1 - 2y_2^1 = 1. \tag{5} $$

B. Planar Coordinates and Ellipse

Choose the center $\mathbf{p}^1 = (-1, -1/2, 0)^T$, the unit plane axes $\mathbf{e}_x = (1, 0, 0)^T$ and $\mathbf{e}_y = (0, 1, 2)^T/\sqrt{5}$, and normal $\mathbf{e}_z = (0, -2, 1)^T/\sqrt{5}$. The matrix ${}^{1}R_p = [\mathbf{e}_x\ \mathbf{e}_y\ \mathbf{e}_z]$ has determinant $+1$, and

$$ {}^{1}T_p = \begin{bmatrix} {}^{1}R_p & \mathbf{p}^1 \\ 0 & 1 \end{bmatrix}, \quad {}^{p}T_2 = ({}^{1}T_p)^{-1}{}^{1}T_2. \tag{6} $$

Applying the inverse transform to AV2's origin gives

$$ {}^{p}\mathbf{o}_2 = \left(\tfrac{1}{2}\sin 2t, \tfrac{\sqrt{5}}{2}\cos 2t, 0\right)^T. \tag{7} $$

Thus $(x_p/(1/2))^2 + (y_p/(\sqrt{5}/2))^2 = 1$. The semi-axis lengths are $1/2$ and $\sqrt{5}/2$. Figure 3 is generated from these equations, not captured from RViz.

IV. Quaternion Properties

The course defines $\Omega_1(q)$ and $\Omega_2(q)$ as the matrices for left and right Hamilton multiplication, using scalar-last vectors: $\Omega_1(a)b = a \otimes b = \Omega_2(b)a$. The quaternion norm is multiplicative, $\|a \otimes b\| = \|a\|\|b\|$. If q is unit, both maps preserve the norm of every vector in $\mathbb{R}^4$. A linear norm-preserving map is orthogonal (by the polarization identity), hence

$$ \Omega_i(q)^T\Omega_i(q) = \Omega_i(q)\Omega_i(q)^T = I_4, \quad i = 1, 2. \tag{8} $$

For $e = (0, 0, 0, 1)^T$, the identity quaternion, $\Omega_i(q)e = q$ for either i. Orthogonality then gives $\Omega_i(q)^T q = e$.

Associativity proves the remaining identities without a 16-entry expansion. For arbitrary x, y, z, $x \otimes (z \otimes y) = (x \otimes z) \otimes y$ implies $\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$. Right multiplication by the conjugate $\bar{y} = (-y_1, -y_2, -y_3, y_4)^T$ is the transpose of right multiplication by y, so $\Omega_2(y)^T = \Omega_2(\bar{y})$. Substituting $\bar{y}$ in the first identity gives $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$. This argument holds even when x and y are not unit.

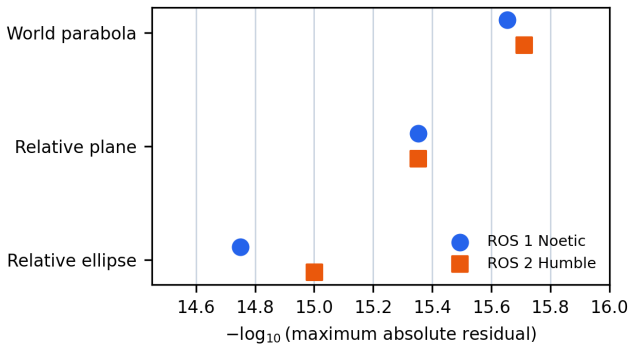


Fig. 4. Precision of the live trajectory checks in both ROS versions. Larger values on the horizontal axis mean smaller maximum residuals. Each result was computed from 30 tf samples; marker counts are reported in Table I.

V. Cloud Experiment and Verification

The Focal environment reported ROS_DISTRO=noetic. The course package compiled both C++ nodes with catkin build -j1; its summary reported two successful targets and zero failures. A separate catkin_make -j1 build passed. A Noetic roscore served the graph; turtlesim ran under Xvfb. Its pose changed from $(5.5444, 5.5444, 0)$ at rest to $(5.1156, 7.4516, -2.7152)$ after the controller published linear $x = 1$ and angular $z = 1$ on /turtle1/cmd_vel. The observed pose velocities were both 1, and the controller exited successfully.

In the Noetic dynamic launch, /frames_publisher_node published /tf, while /plots_publisher_node published /visuals; RViz subscribed to both. A live verifier collected 30 tf samples and 139 MarkerArray messages, each containing two meshes and three trails. Maximum residuals were 2.22×10^{-16} for the AV2 parabola, 4.44×10^{-16} for Eq. (5), and 1.78×10^{-15} for the ellipse. Independently fetched latest transforms differed by up to 0.0426 because their timestamps were asynchronous. The static Noetic launch had two /tf_static publishers. A second verifier measured the expected AV1 and AV2 translations $(1, 0, 0)$ and $(0, 0, 1)$, identity rotations, and five markers.

On the native Ubuntu 22.04 host, the ROS 2 Humble counterpart passed colcon build in 50.9 s. Its dynamic verifier sampled 30 tf transforms and 14 MarkerArrays with five markers each; the plane and ellipse residuals were below 10^{-15}. Its static graph and turtlesim controller also passed. Table I summarizes the run coverage, and Fig. 4 compares the measured residuals. Local checks confirmed the trajectory identities at 1001 times below 10^{-12} residual and quaternion identities on 1000 deterministic samples. Raw cloud logs accompany the deliverables. Analytical plots here come from equations rather than RViz screenshots; the optional rotation-order proof is supplied separately.

TABLE I

Cloud runtime verification

System	tf samples	MarkerArrays	Static check
Noetic	30	139	pass
Humble	30	14	pass

References

[1] Y. Yang, “Visual SLAM / Robot Integration: Experiment II,” course slides, supplied Lab2.pdf.
[2] MIT VNAV, “Lab 2 Exercises,” 2023. [Online]. Available: https://vnav.mit.edu/labs_2023/lab2/exercises.html
[3] MIT-SPARK, “VNAV-labs,” VNAV2023 tag. [Online]. Available: https://github.com/MIT-SPARK/VNAV-labs/tree/VNAV2023/lab2
[4] MIT-SPARK, “VNAV-labs,” ROS 2 main branch. [Online]. Available: https://github.com/MIT-SPARK/VNAV-labs/tree/main/lab2
[5] IEEE Robotics and Automation Society, “(La)TeX Support for manuscript preparation.” [Online]. Available: https://ras.papercept.net/conferences/support/tex.php
[6] ROS Wiki, “noetic/Installation/Ubuntu.” [Online]. Available: https://wiki.ros.org/noetic/Installation/Ubuntu